

Aula 1 (Módulo 10) — Telemetria de EDR: Process Tree, Arquivo, Registry e Rede

1. EDR vs. SIEM — a diferença de foco

O Módulo 09 mostrou o SIEM centralizando logs de **toda** a organização. **EDR** (*Endpoint Detection and Response*) foca fundo em **um único host**, com muito mais detalhe do que um log tradicional oferece — é um **agente** instalado na máquina, observando em tempo real tudo que acontece: criação de processo, acesso a arquivo, modificação de registry, conexão de rede, tudo isso **correlacionado entre si automaticamente**, sem precisar de um SIEM separado para juntar as peças.

```
SIEM → visão AMPLA (toda a organização), baseada em logs já gerados
EDR → visão PROFUNDA (um host), telemetria nativa e correlação local
```

Um ambiente maduro usa os dois juntos: o EDR gera seus próprios alertas ricos localmente, e também **envia** esses dados para o SIEM central, para correlacionar com outras fontes (firewall, proxy, etc.).

2. Process Tree — o coração da telemetria de EDR

Já construímos essa intuição desde o Módulo 00, Aula 1: PID, PPID, usuário, command line. Um EDR mostra isso de forma **visual e navegável** — uma árvore completa, com cada processo podendo ser expandido para ver seus próprios filhos.

```
explorer.exe (PID 892, usuário: ana)
├─ winword.exe (PID 3102)
│   └─ powershell.exe (PID 4821) ← ALERTA: filho inesperado de Word
│       └─ cmd.exe (PID 5011)
│           └─ whoami.exe (PID 5033)
```

Por que importa: um EDR não olha só "o processo é malicioso?" isoladamente — ele avalia o **caminho inteiro** até a raiz. Um `whoami.exe` sozinho é absolutamente inofensivo (é um comando comum de diagnóstico); o mesmo `whoami.exe`, filho de uma cadeia que começou em `winword.exe → powershell.exe → cmd.exe`, é um forte indício de **reconhecimento pós-exploração** — um atacante confirmando com qual usuário/privilegio ele conseguiu executar código.

3. Command line completa — o que realmente foi executado

Além do nome do processo, o EDR captura a **command line inteira**, incluindo argumentos — exatamente o que discutimos no Módulo 02, Aula 2, sobre `-enc`, `-WindowStyle Hidden`, `-ExecutionPolicy Bypass`.

Isso é crítico porque dois processos com o **mesmo nome** podem ter propósitos completamente diferentes dependendo dos argumentos:

```
rundll32.exe C:\Windows\System32\shell32.dll,Control_RunDLL ← normal
rundll32.exe javascript:"..\mshtml,RunHTMLApplication ";... ← técnica de execução via LOLBins
```

(`rundll32.exe` sendo usado para executar código através de uma técnica conhecida como **LOLBin** — *Living Off the Land Binary*, o mesmo espírito de "living off the land" já mencionado no Módulo 02, Aula 2 sobre PowerShell — usar programas legítimos do próprio Windows para fins maliciosos, dificultando a detecção baseada só em "esse programa é malicioso ou não".)

4. Atividade de arquivo

O EDR registra criação, modificação, renomeação e exclusão de arquivos — muito parecido com o Sysmon Event ID 11 que vimos no Módulo 02, Aula 5, mas com contexto adicional automático: **qual processo** criou o arquivo, **onde** (pastas incomuns como `%TEMP%` são sinalizadas com mais peso), e às vezes até o **hash** calculado automaticamente no momento da criação.

Por que importa: ransomware, por exemplo, gera um padrão de atividade de arquivo extremamente característico — **muitos arquivos sendo modificados/renomeados em sequência muito rápida**, geralmente com uma nova extensão adicionada ao final. EDRs modernos têm detecção específica só para esse padrão de comportamento, independente de conhecer a assinatura daquele ransomware específico.

5. Atividade de Registry

Lembra das Run keys do Módulo 02, Aula 1 (`...\CurrentVersion\Run`)? O EDR monitora escritas nessas chaves (e em muitas outras usadas para persistência) em **tempo real**, permitindo alertar no exato momento em que algo tenta se estabelecer como persistente — antes mesmo do próximo reboot acontecer.

6. Atividade de rede

O EDR também vê conexões de rede feitas por cada processo — parecido com o Sysmon Event ID 3 (Módulo 02, Aula 5) e com o `ss -tulnp` que vimos no Módulo 01, mas já correlacionado automaticamente com o processo, o usuário, e o resto da árvore. Muitos EDRs também mantêm uma base própria de **reputação de IP/domínio** (conectando com Threat Intelligence, Módulo 12), sinalizando conexões para infraestrutura maliciosa conhecida no momento em que acontecem.

7. Conectando tudo — como o EDR "pensa"

Process tree	→ contexto de ORIGEM (de onde veio esse processo?)
Command line	→ contexto de INTENÇÃO (o que ele está tentando fazer?)
Arquivo	→ contexto de IMPACTO (o que ele está criando/mudando?)
Registry	→ contexto de PERSISTÊNCIA (ele quer sobreviver a um reboot?)
Rede	→ contexto de COMUNICAÇÃO (ele está falando com alguém de fora?)

Um bom analista de EDR aprende a olhar essas cinco dimensões **juntas** para qualquer processo suspeito — nenhuma sozinha conta a história completa, mas todas juntas normalmente deixam claro se algo é malicioso.

8. Exercício de reflexão

Usando só as cinco dimensões acima, monte mentalmente o "perfil" de telemetria que você esperaria ver para cada um destes cenários (sem resposta pronta — pense em cada dimensão):

1. Um ransomware em atividade.
2. Uma tarefa agendada maliciosa recém-criada, ainda não executada.
3. Exfiltração de dados através de uma conexão HTTPS de saída.

9. Recapitulando

- EDR foca profundamente em um host, com telemetria nativa e correlação automática — complementa (não substitui) o SIEM.
- Process tree revela a origem real de um processo, mesmo quando o nome do processo em si parece inofensivo.
- Command line completa diferencia uso legítimo de abuso do mesmo binário (LOLBins).
- Arquivo, registry e rede completam o quadro: impacto, persistência e comunicação externa.

Próxima aula (fecha o Módulo 10)

Investigações simuladas de EDR — aplicando as cinco dimensões desta aula em cenários reais: PowerShell suspeito, execução de binário incomum, processo filho anômalo, persistência e conexão suspeita.

Fontes recomendadas

- **MITRE ATT&CK — "Living off the Trusted Land" e T1218 (System Binary Proxy Execution)**: técnica oficial associada ao abuso de LOLBins como `rundll32.exe`. <https://attack.mitre.org/techniques/T1218/>
- **LOLBAS Project (lolbas-project.github.io)**: catálogo comunitário amplamente referenciado na indústria de binários legítimos do Windows abusáveis para execução/evasão — fonte comunitária, não oficial de nenhum fabricante, mas citada aqui por ser referência padrão da área. <https://lolbas-project.github.io/>